olive-solve fork vs. `original`, and the extractor landscape

Date: 2026-07-21 **Scope:** Two questions for the diofinder ecosystem —

1. How does *your* olive-solve (the `main` line diofinder consumes as its plate **solver**) differ from olive-solve's `original` branch?
2. How do the four candidate **centroid extractors** — `sycamore-extract`, `cedar-detect`, olive-solve's **original extractor**, and the **tetra3rs** approach — differ, and which belongs where?

Sources: this document consolidates and updates prior in-repo assessments (`docs/decisions/2026-06-27-olive-solve-original-branch-assessment.md`, `docs/decisions/2026-06-12-sycamore-only-pipeline.md`, the `*20260612*` build-configuration records, and the project `CLAUDE.md` / `ARCHITECTURE.md` files) with a fresh read of the current sources (`olive-solve/tetra3/src/`, `sycamore-extract/src/lib.rs`, `cedar-detect/src/algorithm.rs`, `tetra3rs/src/centroid_extraction.rs`).

Executive summary

- **olive-solve `main` (your fork) is strictly the right choice for diofinder, and `original` is a regression to adopt wholesale.** The two branches *diverged*; they are not older/newer versions of one another. `main` carries the attitude-hint / blind-fallback API that diofinder's tracking and IMU-hint solving *depend on*, plus the f32 memory work and rayon multi-core parallelism that matter on the Pi Zero 2W. `original` has none of these.
- **The valuable algorithmic ideas that were unique to `original` have since been ported onto `main`** (olive-solve v0.1.6 `verify_attitude`, v0.1.7's five micro-optimizations). So the earlier "port the early-rejection SVD pre-pass" recommendation is now **done**, and the gap has effectively closed in `main`'s favour. `original`'s later addition — a standalone `olive-imu` crate — remains a **skip** (wrong sensor, redundant with diofinder's own Kabsch fit).
- **On extraction, the four contenders occupy different niches, and diofinder's current split is correct:** `sycamore-extract` is the live finder extractor (matched filter + temporal cache, u8, in-process, ROI tracking); olive-solve's extractor is kept only as the **"Legacy" A/B baseline**; `tetra3rs`'s extractor is the right tool **off-device** for SIP-distortion lens calibration; and `cedar-detect` has been **fully removed** — its one borrowable idea (perimeter/annulus local noise) was already reimplemented in sycamore. Nothing argues for reintroducing it.

Part A — olive-solve: your fork (`main`) vs. the `original` branch

A.1 What diofinder actually consumes

diofinder uses olive-solve as its **plate solver**: `solver_proc.py` calls `tetra3.Tetra3.solve_from_centroids` (the Rust engine in `tetra3/src/solver.rs`). The release pipeline pulls the wheel from olive-solve's GitHub releases, which are cut from `main`. So any change to `solver.rs` on `main` lands directly in diofinder's live solve path.

Note the naming: olive-solve publishes the Python package as `tetra3` (distinct from the separate `tetra3rs` PyPI project). Both descend from the ESA Tetra3 / Cedar-Solve algorithm lineage (Steven Rosenthal), but they are different codebases.

A.2 How the branches relate

```
                452602a  (merge-base: "block median … python fast path")
               /        \
original (+3 perf,      main (+~35: f32, rayon, hint API, …)
later +olive-imu)             |
235a5d9                       └── releases ──► diofinder wheel
```

`original` and `main` **diverged from a common merge-base** and rewrote the *same* hot code (`verify_and_build_solution`, the hash-probe, the 4-star combinatorics loop) in **different, non-cherry-pickable** ways. `original` is a short perf-focused line (three "native optimization" commits, later joined by an IMU crate); `main` is the long-lived line diofinder tracks.

A.3 What `main` has that `original` does not

These are the strategically important wins, already live in diofinder:

Capability	Why it matters for diofinder	On `original`?
Attitude-hint / blind-fallback API (`attitude_hint` , `hint_uncertainty_deg` , `strict_hint` , blind retry)	`tracking.py` (tight 2° cone, `strict_hint=True`) and IMU-propagated hint solves **require** it; its absence caused a fleet re-acquisition deadlock historically	**No**
f32 resident catalog + KdTree (`[f32;3]` vectors, `KdTree<f32,3>` , f32 verify scratch)	Halves the resident catalog/KdTree footprint and the dominant neighbour-query bandwidth on the memory-bandwidth-bound 512 MB A53	No (all f64)
rayon multi-core candidate search (`par_chunks(…).find_map_first`)	Uses diofinder's three dedicated solver cores (CPUs 1–3)	No (single-threaded)
cortex-a53 `target-cpu` pinning	Matches the deployed silicon	(via build config only)

Because of the first row alone, **diofinder cannot move to `original` without losing tracking and hinted solving.** That rules out a branch switch on its own.

A.4 What `original` had that `main` lacked — and the current status

The earlier assessment identified a set of algorithmic optimizations unique to `original` and recommended porting the best of them onto `main` . **That work has since happened.** Current status:

`original` idea	Value	Status on `main`
Early-rejection SVD pre-pass (cheap 3×3 dot-product gate before the heavy KD-tree verify)	Highest	**Ported** (v0.1.7, PR #21)
Lazy / streaming verification with early break (stop at `2·num_extracted` kept stars)	Good	**Ported** as "lazy verification early-break" (v0.1.7)
Direct 2×2 Cramer's-rule distortion refine (replaces nalgebra SVD pseudo-inverse)	Modest	**Ported** (v0.1.7)
Reciprocal-multiply / loop-invariant hoisting in the edge-ratio loop	Low	**Ported** (v0.1.7)
`ImmutableKdTree` (build-order ids, `within_unsorted`)	Modest	**Ported at f32** — kept f32, unlike upstream's f64 (v0.1.7)
Fixed 12-comparator sorting network for the 6 edges	Low (rayon already spreads this)	**Deliberately not ported** (documented in AGENTS.md backlog)
Monotonic key-space pruning of the hash probe	Low/uncertain	**Deliberately not ported**
Probe table + `usize` → `u32` pattern-catalog packing	Low (superseded by f32)	Not ported; open isolated micro-win
In-solver watchdog thread → inline timeout check	Optional	Not ported (diofinder has its own comms-side watchdog)

Separately, `main` added a capability neither branch had before: **`verify_attitude`** (v0.1.6, PR #19) — a verify-only fast path that projects the catalog through a caller-supplied quaternion and skips the 4-star pattern hash entirely. This is what unlocks diofinder's tracking fast path (measured ~0.01 ms vs ~0.5 ms for a full solve, identical RA/Dec).

Net: the high-value content of `original` now lives in `main` , re-expressed in `main` 's f32 + per-worker-scratch parallel structure and validated for solve rate (not just latency). The remaining `original` -only items are explicitly-declined low-value micro-ops.

A.5 The `olive-imu` crate on `original` — skip

`original` later grew a standalone **`olive-imu`** Rust crate (~1,900 LOC, author oakamil): I2C drivers for **BMI160 + BNO085**, real-time SVD camera↔IMU frame alignment, continuous gyro-bias compensation, and a 100 Hz+ async (`tokio`) polling thread. For diofinder this is a **skip**:

- **Wrong sensor.** diofinder is committed to the **BNO055** (`imu_proc.py`), which fuses on-chip and returns a ready quaternion. olive-imu's continuous bias compensation exists precisely because a raw IMU (BMI160) has no onboard fusion — moot when the chip already fuses.
- **Redundant.** olive-imu's "real-time SVD alignment" is the same math as diofinder's **Kabsch fit** (`imu_frame.py`) — but diofinder's is quality-gated (≥4 magnitude-consistent pairs, axis diversity ≥0.25, $R^2 \geq 0.9$, refuses unobservable alt-only slews) and feeds a tear-proof

`imu_ref` + exact quaternion LX200 prediction with kill switches. diofinder's is field-hardened; olive-imu's is generic and new.

- **Wrong architecture.** Standalone Rust + `tokio`, not part of the `tetra3` wheel, no Python bindings — adopting it means PyO3 packaging plus rewriting the comms-process IMU integration. High cost, negative net.

The one borrowable idea — back-dating IMU timestamps to the true measurement instant — buys little at diofinder's deliberate 20 Hz poll (50 ms granularity dominates the ~1 ms I2C jitter), and its main consumer (the inter-solve Kabsch fit over seconds) is insensitive to a 1 ms stamp error.

A.6 Verdict (Part A)

Stay on `main`. Do not switch to `original`. `main` is not merely newer — it is the only branch with the hint/tracking API diofinder is built around, it carries the f32 + rayon wins that matter on the Zero 2W, and it has since absorbed the worthwhile algorithmic optimizations that were once unique to `original`. `original`'s remaining distinctive content (the `olive-imu` crate) is a deliberate skip.

Part B — the extractor landscape

Four centroid extractors are in play. They share a common ancestry (ESA Tetra3 / Cedar) but were built for different priorities. The table first, then each in turn, then how diofinder uses them.

B.1 Side-by-side

Dimension	sycamore-extract (`star_detect`)	cedar-detect	olive-solve extractor (`FastExtractor` / `get_centroids_from_image_fast`)	tetra3rs (`centroid_extraction`)
Language / form	Rust PyO3 wheel (in-process)	Rust; **gRPC server binary** (separate process)	Rust; in-process (inside the `tetra3` wheel)	Rust; in-process (inside the `tetra3` solver crate, feature `image`)
Author / lineage	Purpose-built for the diofinder finder	Steven Rosenthal (Cedar), Apache-2.0 — the reference	AstroKeith's tetra3 port ("classic tetra3" defaults)	Independent Rust reimpl of Tetra3/Cedar, astrometry-focused
Detection gate	**1-D matched filter** (integer, mean-zero kernel cancels DC), runtime `kernel_sigma`	Localized thresholding, multi-pixel evidence	`background + sigma·noise` threshold on background-subtracted image	`background + sigma·noise` , optional matched-filter pre-blur
Background model	Per-frame (7 modes: row/line/column/block/uniform/top-hat) **+ temporal "analytic-threaded" cache**	Localized (adapts across the image)	**LocalMean** (25-px sliding window)	**Sigma-clipped median** (global) + **per-blob annulus** local background
Noise estimate	MAD (default) or global-RMS; **perimeter/ring local-noise inflation**	Adaptive per-image noise estimate	**GlobalRootSquare** (matches classic tetra3)	Sigma-clipped background σ
Sub-pixel	Matched-filter peak / centroid	Centroid of candidate	Intensity-weighted centroid (i16 fixed-point, 7-bit subpixel)	**Quadratic peak refinement**
Pixel depth	**u8 only**	u8 (native camera resolution)	u8 zero-copy (i16 fixed-point internal)	f32 throughout
Trail rejection	Full **2-D second-moment** axis-ratio	Yes (trailed-object rejection)	`max_axis_ratio` (separable)	(via blob shape)
Hot-pixel	Via diofinder `bg_cache` + static mask	**Built-in** classify/reject	None	(blob min-area)
Distortion / astrometry	None (finder-grade: "within a pixel or two of FOV centre")	None (detector only)	None	**SIP polynomial +** `calibrate_camera`
Concurrency	Bounded thread pool (2 default; **3** in diofinder), GIL released	Server-side	Pre-allocated buffers, single-pass	rayon (`parallel` feature)
Perf note	p50 **< 6 ms** at bin=2 on Zero 2W (the regression gate)	~<10 ms / 1M px on Pi 4B	Integer pipeline, halves memory bandwidth	`estimate_local_background` ≈ 60% of wall-clock
Output convention	`(x=col, y=row)` → **needs swap** for the solver	pixel coords	`[row, col]` directly (no swap)	origin-at-image-**centre** (solver-native)
Role in diofinder	**Live extractor (shipped)**	**Removed**	**"Legacy" A/B baseline**	**Off-device lens calibration only**

B.2 cedar-detect — the reference ancestor (removed from diofinder)

`cedar-detect` (Steven Rosenthal, Apache-2.0) is the canonical detector of this family: a single efficient pass generates a few hundred/thousand candidates, then localized thresholding, adaptive noise estimation, built-in hot-pixel classification, trailed-object rejection, and tolerance of bright interlopers (moon, streetlights). It is genuinely good and fast (~<10 ms/1M px on a Pi 4B).

Its limitation for diofinder is **architectural, not algorithmic**: cedar-detect ships as a **gRPC server binary** with no in-process Python link. In the finder that meant a separate process plus an image→centroids IPC/serialization round-trip on every solve — the dominant overhead the pipeline was trying to kill. The `2026-06-12-sycamore-only-pipeline` decision **removed cedar-detect entirely** (server, `.proto` , systemd unit, gate mode, all references). The one idea worth keeping — perimeter/annulus **local-noise** estimation — was **independently reimplemented** in sycamore (`local_noise` , credited as concept-inspired-by-cedar). There is no remaining reason to reintroduce the gRPC detector.

B.3 olive-solve's extractor — the "classic tetra3" baseline

olive-solve carries tetra3's own extractor (`Extractor` / the integer-optimized `FastExtractor` , exposed to Python as `get_centroids_from_image_fast`). Its defaults *are* the classic tetra3/Cedar pipeline: **LocalMean** background (`filtsize=25` sliding window) + **GlobalRootSquare** noise + sigma threshold + `binary_open` , with `min_area=5` , `max_area=100` . `FastExtractor` is a zero-copy u8 integer pipeline that stores background-subtracted intensities as i16 scaled by 128 (7 bits of sub-pixel precision) and pools downsampled pixels as u32 — explicitly to halve memory bandwidth on the Pi.

In diofinder this is exactly the **"Legacy" seeing preset** (`extractor_backend = "tetra3"`): an exact re-creation of the AstroKeith `eFinder_cli` `original` -branch pipeline (`downsample=1` , `min_area=5` , `max_area=100` , no matched filter, no temporal cache, no hot-pixel). It returns `[row, col]` directly (no x/y swap, unlike sycamore). It is capability-probed and kept as the **baseline to improve upon** for clean A/B comparison — not the recommended default.

B.4 tetra3rs's extractor — the astrometry-grade path

`tetra3rs/src/centroid_extraction.rs` is a from-scratch Rust reimplementation oriented toward **astrometry**, not just finding: sigma-clipped **median** background estimation (5 iterations, 3σ clip), connected-component labeling of blobs, a **per-blob local background from an annulus of non-blob pixels**, **quadratic sub-pixel** peak refinement, and an optional matched-filter pre-blur (`matched_filter_sigma`). It runs in **f32** end-to-end and is the front end of a full solver crate that also does **SIP polynomial distortion** and multi-image `calibrate_camera` .

That accuracy focus is also its cost: `estimate_local_background` alone is ~60% of extraction wall-clock, and f32 doubles the memory traffic the finder works hard to avoid. It is heavier than the finder needs for "within a pixel or two of the FOV centre." diofinder uses tetra3rs **off-device**: `scripts/calibrate_lens.py` runs `tetra3rs` `calibrate_camera` on a directory of solved PNGs to fit the SIP `distortion` value for `diofinder.conf` . It is the right tool for that job and the wrong tool for the hot path.

B.5 sycamore-extract — the purpose-built finder extractor (shipped)

`sycamore-extract` (`star_detect`) is the extractor diofinder actually ships. It is deliberately **only** an extractor (solving is delegated to olive-solve), and every load-bearing decision is tuned for a Pi Zero 2 W finder:

- **Single 1-D matched-filter gate**, integer arithmetic, mean-zero kernel so DC cancels (no per-pixel background subtraction needed for the gate); runtime `kernel_sigma` (1.0–4.0) to match bloated PSFs.
- **u8-only** — the finder never needs the 12-bit range, which halves bandwidth.
- **Temporal "analytic-threaded" background cache** — a worker thread median-stacks recent frames into a per-row/per-block/per-pixel model, giving √N noise reduction and free hot-pixel rejection in steady state, and falling back to per-frame estimation during slew/warm-up. This is the architectural advance the finder is built around and the reason per-frame detectors (cedar/tetra3) are not competitive here.
- **Seven per-frame background modes**, full **2-D second-moment** trail rejection, **perimeter-derived local noise** (the borrowed cedar idea), and a native **ROI window** path (`detect_stars_roi`) for tracking mode.
- Bounded thread pool (3 on the diofinder deployment), GIL released during detection, p50 **< 6 ms** at bin=2 as a hard regression gate.

Its one honest caveat: the matched-filter threshold assumes Gaussian noise, so on real correlated-noise sky it is somewhat conservative — users lower `sigma` 1–2 from the conventional default to reach faint stars. That is a tuning knob, not a design flaw.

B.6 How they relate, in one line each

- **cedar-detect** — the fast, feature-complete *reference* detector, but gRPC-only → removed from the finder for IPC cost; its local-noise idea lives on in sycamore.
- **olive-solve extractor** — the *classic tetra3* pipeline (LocalMean + GlobalRMS), integer-optimized; diofinder keeps it as the **Legacy A/B baseline**.
- **tetra3rs extractor** — the *astrometry-grade* path (sigma-clipped median + CCL + annulus + quadratic + SIP); used **off-device for lens calibration**.
- **sycamore-extract** — the *purpose-built finder* extractor (matched filter + temporal cache + ROI, u8, in-process); the one diofinder **ships**.

B.7 Recommendations (Part B)

1. **Keep sycamore as the live extractor.** Matched filter + temporal cache + in-process u8 is the correct architecture for this finder; nothing in the other three beats it on the Zero 2W hot path.
2. **Keep the Legacy (olive-solve/tetra3) extractor as the A/B baseline only.** It is the honest "classic tetra3" reference to measure sycamore against; do not promote it to default.

3. **Do not reintroduce cedar-detect.** Its architectural cost (separate gRPC process + per-solve IPC) is exactly what was removed; its one useful idea is already reimplemented.
4. **Keep tetra3rs off-device for SIP lens calibration.** It is the right tool for `calibrate_camera` and the wrong tool for live extraction (f32 weight, ~60% local-background cost).

Bottom line

- **Solver:** your olive-solve `main` fork is the correct dependency and is now strictly ahead of `original` for diofinder — it owns the hint/tracking API, the f32 + rayon Pi wins, and (since v0.1.6/v0.1.7) the worthwhile algorithmic optimizations that were once unique to `original`. `original`'s `olive-imu` crate is a deliberate skip.
- **Extractor:** diofinder's current arrangement is right — sycamore live, Legacy/tetra3 as the A/B baseline, tetra3rs for off-device calibration, and cedar-detect retired. No change recommended.